This is just the leapfrog/Verlet method in its Hamiltonian form. Since we have obtained the method as the composition of two symplectic maps, and the symplectic maps form a group, we know that this method will also be symplectic.

When we construct a composition of a method with its adjoint method, we see readily that it is symmetric, i.e. self-adjoint, since

$$\mathcal{K}_h^\dagger := [\mathcal{G}_{h/2}^\dagger \circ \mathcal{G}_{h/2}]^\dagger = \mathcal{G}_{h/2}^\dagger \circ \mathcal{G}_{h/2} = \mathcal{K}_h.$$

Self-adjoint, or *symmetric* schemes have unique features; in particular they have even order [164, 227].

As we shall see in the next chapter, it is possible to construct methods of arbitrary order by employing more sophisticated multi-stage compositions of mappings.

2.4.2 General Composition Methods

When using splittings, it is not necessary to solve each Hamiltonian of a splitting using the exact flow. Instead, we may replace the flow maps of any part by an approximation. More generally, if we have any two symplectic numerical methods, say $\mathcal{G}_h^1$ and $\mathcal{G}_h^2$, then the composition

$$\mathcal{G}_h := \mathcal{G}_{h/2}^1 \circ \mathcal{G}_{h/2}^2$$

is another symplectic numerical method. The order of this method is typically the minimum of the orders of the two methods involved, but it can be higher, as the example of the Verlet method (constructed by composing symplectic Euler and its adjoint) shows.

2.4.3 Harmonic + Anharmonic Splitting

Some systems can be decomposed into a harmonic (quadratic) part $H_0(\boldsymbol{q},\boldsymbol{p}) = \boldsymbol{p}^T\boldsymbol{M}^{-1}\boldsymbol{p}/2 + \boldsymbol{q}^T\boldsymbol{A}\boldsymbol{q}/2$ and an anharmonic part $H_1(\boldsymbol{q},\boldsymbol{p}) = \hat{U}(\boldsymbol{q})$. Then a splitting method can be formulated based on exact solution of the linear system together with "kicks" representing impulses defined by the anharmonic part. For a simple system consisting of a harmonic oscillator $H_0(q,p) = p^2/2 + \Omega^2 q^2/2$, such a method can be written

$$\begin{bmatrix} Q \\ \hat{p} \end{bmatrix} = \begin{bmatrix} \cos h\Omega & \frac{1}{\Omega}\sin h\Omega \\ -\Omega \sin h\Omega & \cos h\Omega \end{bmatrix} \begin{bmatrix} q \\ p \end{bmatrix},$$
$$P = \hat{p} - h\hat{U}'(Q).$$

2.4.4 Explicit and Implicit Methods

All of the methods mentioned so far are said to be *explicit* discretizations, since producing the next approximation point on a trajectory does not require solving any implicit equations defined in terms of the previous one. The Verlet method is not implicit even though $\boldsymbol{Q}$ appears on the right in the second equation, since it is given in an explicit way in terms of $\boldsymbol{q}$ and $\boldsymbol{p}$. Implicitness adds another layer in both analysis and numerical implementation, which, however, in certain applications, is readily justified.

Example 2.6 The *Backward Euler* method,

$$z_{n+1} = z_n + h\boldsymbol{f}(z_{n+1}),$$

is an example of an implicit method. The calculation of a timestep involves solving a system of equations of the form

$$\boldsymbol{g}(\boldsymbol{w}) \equiv \boldsymbol{w} - z_n - h\boldsymbol{f}(\boldsymbol{w}) = 0.$$

The map $\mathscr{G}_t$ is defined implicitly by the equation

$$\mathscr{G}_h(z) = z + h\boldsymbol{f}(\mathscr{G}_h(z)).$$

An implicit method will typically result in a system of nonlinear equations of the form

$$\boldsymbol{g}(z_{n+1}) = \boldsymbol{\tau}_n, \tag{2.25}$$

which will need to be solved at each timestep. The right hand side $\boldsymbol{\tau}_n$ is a vector that depends on the previous time-step z_n, perhaps in a complicated way. We may assume the number of equations represented by (2.25) is equal to the dimension of the space where z is defined, so we have a square nonlinear system. Typically $\boldsymbol{g}$ will depend on the stepsize and coefficients of the method and will have the property that for h sufficiently small, the solution is uniquely defined and is continuously defined in terms of $\boldsymbol{\tau}_n$, that is the mapping $\boldsymbol{g}$ has a bounded and smooth inverse.

Solving the system may then proceed, from an initial guess $z_{n+1}^{(0)}$ by use of Newton's method:

$$z_{n+1}^{(k+1)} = z_{n+1}^{(k)} - \left[\boldsymbol{J}^{(k)}\right]^{-1} (\boldsymbol{g}(z_n^{(k)}) - \boldsymbol{\tau}_n),$$

where $\boldsymbol{J}^{(k)}$ is the Jacobian matrix of the mapping $\boldsymbol{g}$ evaluated at $z_n^{(k)}$, or else an approximation of this Jacobian (assumed to be nonsingular due to the invertibility